

Code for 2D Graphics



We can easily create graphics on a paper or using a software. But do you know how we can create graphics with codes?



Our world surrounds us with computers. Devices like smartphones, tablets, smart TVs, and laptops are all types of computers. But devices like washing machines, refrigerators, ovens, car infotainment systems, and ATM machines have computers inside them too.



7. Code for 2D Graphics

Engineers create a set of instructions for devices, which makes them perform tasks in a certain way. A set of instructions like this is called a program or software.

Coding

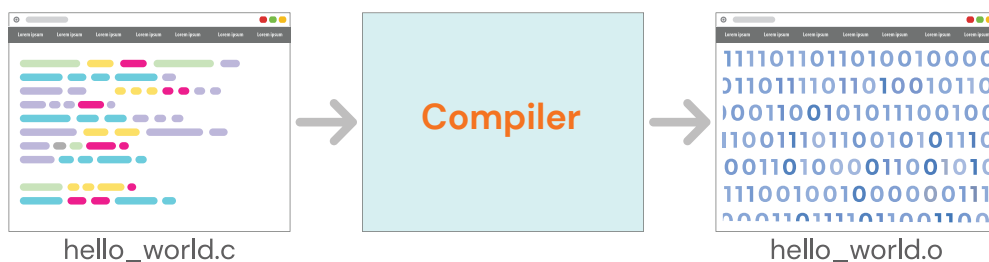
A code or program is a set of instructions written in a specific format. A computer instruction format is based on different rules and syntax. This is called a programming language.

For example: C++, JavaScript, Python, Java.



In English, Hindi, French, German, Japanese or any other language, the same sentence will have a different script and pronunciation. A computer's language is the same way, and is called the **Binary Language**. A computer only understands a language made up of '0's and '1's. The script looks something like this: '001011001111100'.

It is difficult for us to write or understand this language directly in 0s and 1s. To overcome this, we write instructions using a programming language. A special program called a 'Compiler' translates code into binary language.



Compiler

A compiler is a special program that converts code from a particular programming language into a machine language that computers understand.

Note:

In next few lessons, we will learn and use the programming language called **JavaScript**.

7. Code for 2D Graphics

JavaScript

JavaScript is a widely used programming language. It is simple to learn and has a vast variety of features and applications.

JavaScript is used to create interactive websites, dynamic webpages, animations, and web-based games. All major websites like Google, YouTube, Facebook and Amazon use JavaScript.



Bytes

Java vs JavaScript: Java and JavaScript have similar names but are very different programming languages. Each programming language has different rules, syntax, and features to code and cater to various applications.

Exercise

State whether True or False

- 1 Computers can only understand the language of 0s and 1s.
▲ _____
- 2 A compiler converts written code into a language that a computer understands.
▲ _____
- 3 Java and JavaScript are the same programming language.
▲ _____

p5.js

p5.js is a free JavaScript platform for creative coding. It makes coding accessible to all ages and is suitable for designers, educators, artists, other professionals, and beginner coders.

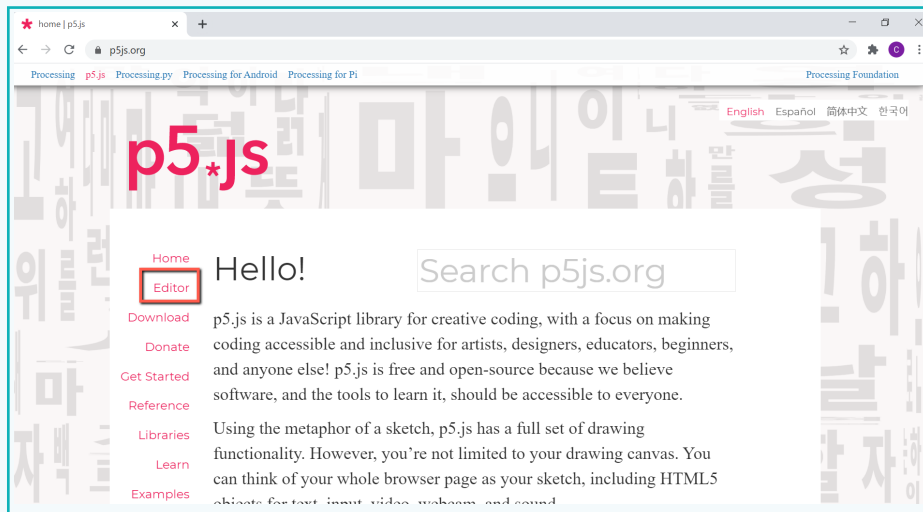


We will use the p5js online editor throughout this course to write code and observe the output. Visit the website <https://p5js.org/> to explore the editor.

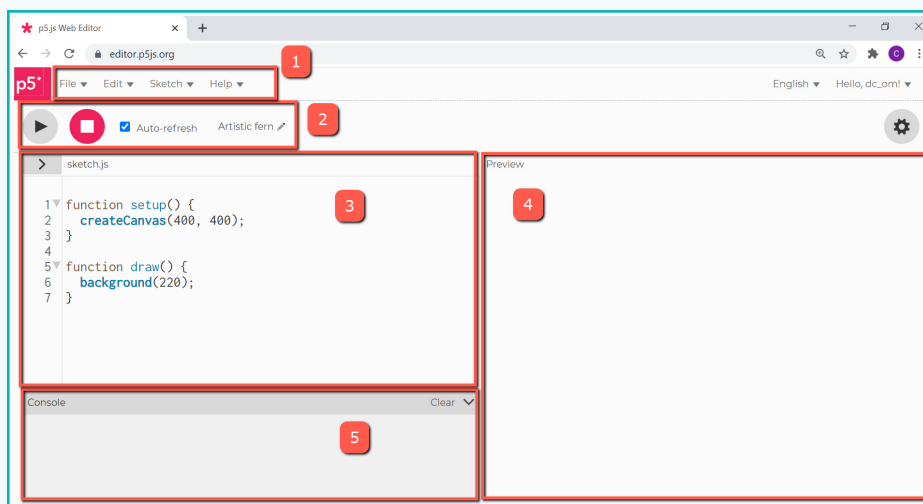
Activity

In this activity, you will learn about the p5js editor and its interface.

- 1 Open the website <https://p5js.org/> and click on Editor.



An editor window will appear with some existing code. Let's understand the interface and the code. Each code we write is called a **sketch**.



7. Code for 2D Graphics

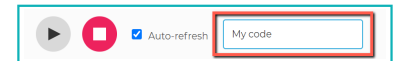
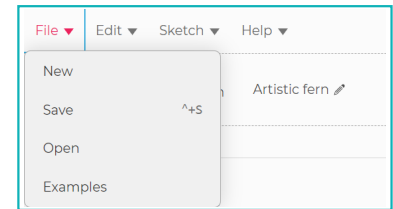
1. Tool bar:

The tool bar provides options to create a new sketch, save it or open existing ones.

2. Dashboard:

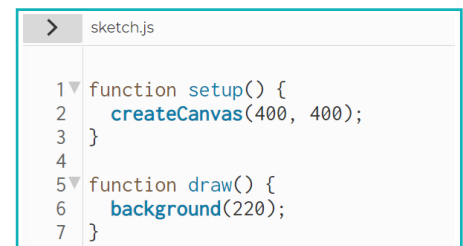
The dashboard has a play button to run the code and a stop button to stop it. When we check the auto refresh box, the code will continue to run and any changes in it will reflect in the output.

- Every time we open the code editor, the file gets a default name. We can click on the pencil-shaped symbol to change the file's name.



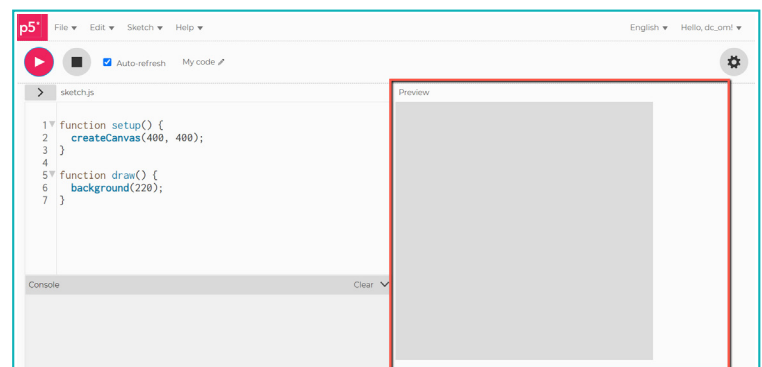
3. Sketch:

This is the editor area where the default code is written.



4. Preview:

This window will show the output of the code we have written. If we click on the default code, a grey square will appear in the preview window.



5. Console:

This area uses specific commands to display program information in the background while it's running. This helps to find errors (or 'bugs') in the code. The process of finding errors in a code is called debugging.

Basic Code

Let's look at the default code that appears onscreen when we open the editor.

```
function setup() {  
  createCanvas(400, 400);  
}  
  
function draw() {  
  background(220);  
}
```

a. Function:

Function is a set of code which has specific tasks allocated to it. Every function starts with the word 'function' followed by the name of the function, a pair of round brackets and a pair of curly brackets. The curly brackets contain the code.

```
function functionName () {  
    //We write code here  
}
```

b. Setup:

Setup() is a default function in sketch that runs once when the code runs.

```
function setup() {  
      
}
```

- Code inside curly brackets defines initial environment properties like screen size and background colour. It also gives instructions to load media like images and fonts when the program starts.

c. Draw:

Draw() is a default function which runs directly after setup(). The code written inside the draw() function runs repeatedly on loop until the program stopped.

```
function draw() {  
      
}
```

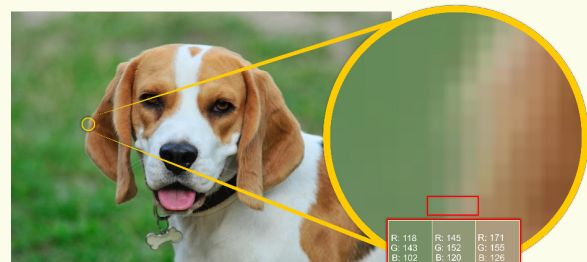
- There can only be one setup() and draw() function in each program.



Recall:

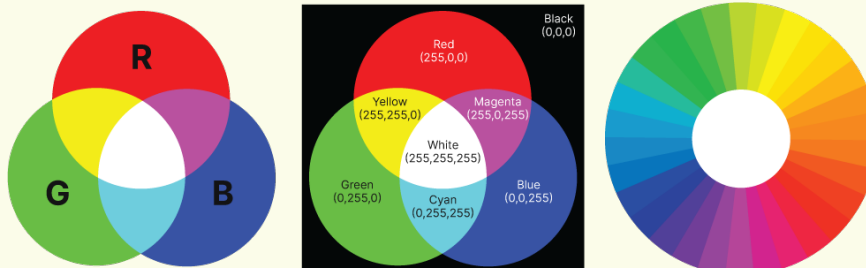
We hear phrases like '13-megapixel camera', or '1028 x 760 pixels screen'. What does this word "pixel" mean?

Pixel: A pixel is a coloured dot or the smallest portion on the screen of a digital display. Pixels are tiny and high in number. When displayed on a screen, they appear to merge into a smooth image.



Canvas: A digital canvas is the space on which a code's output is "drawn" or displayed. It is similar to the canvas we use to paint on.

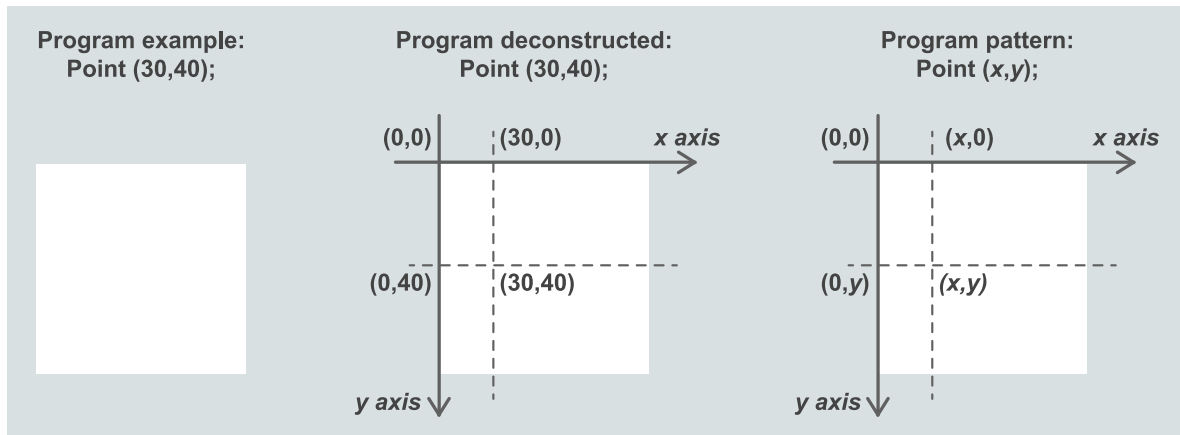
RGB colours: Computers understand and use colours to display the screen in an RGB format. RGB stands for RED: GREEN: BLUE. These three colours are mixed in various intensities to get different colours.



Any colour can be represented with a combination of these 3 colours on the scale of 0 to 255, with 255 denoting the highest intensity.

When we create a canvas, we use pixels to define width and height. Each point on the canvas can be represented by its x and y position. We need to know the x and y position of a specific point on the canvas to draw different shapes.

For example: A point can be drawn at a location on the canvas with x and y positions as shown in the images below.



Exercise

Tick mark ☒ the correct answer

- 4 _____ is a default function that runs only once at the beginning of the code execution.

a. Setup	☐	b. Draw	☐
c. Both A and B	☐	d. None of the above	☐

- 5 The size of the canvas in the output is measured in _____.

a. Pixels	☐	b. cm	☐
c. mm	☐	d. feet	☐

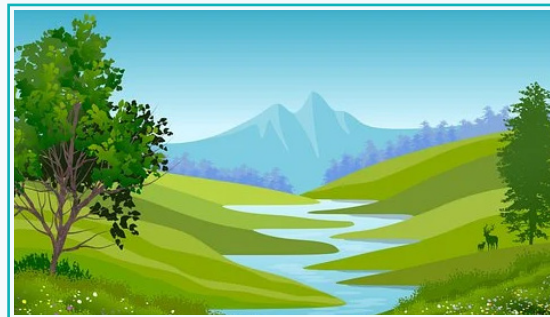
- 6 _____ is a group or set of code that has a specific task and can be called when required in the code execution.

a. Function	☐	b. Pixel	☐
c. Color	☐	d. Compiler	☐

Activity

Think about the drawings you made as a child. All of us have drawn clouds, mountains, streams, and trees.

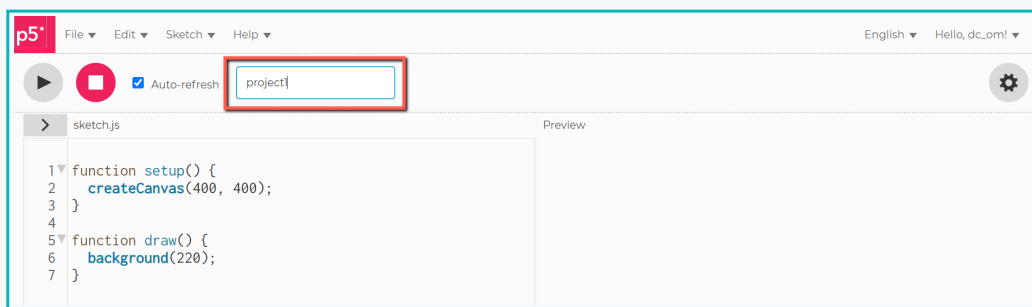
For example:



In this lesson, we will use basic shapes to create a scenic landscape with JavaScript.

2 Open p5js editor

a. Open the editor. A default code will appear. Change the file name to project1.



b. **createCanvas()**: This command creates a canvas with dimensions in pixels. This should be called only once at the start of the setup function.

c. **background(R,G,B)**: This function sets the background colour for the canvas. The default background is transparent. It takes values red, green and blue on the scale of 0 to 255 to create a range of colours.

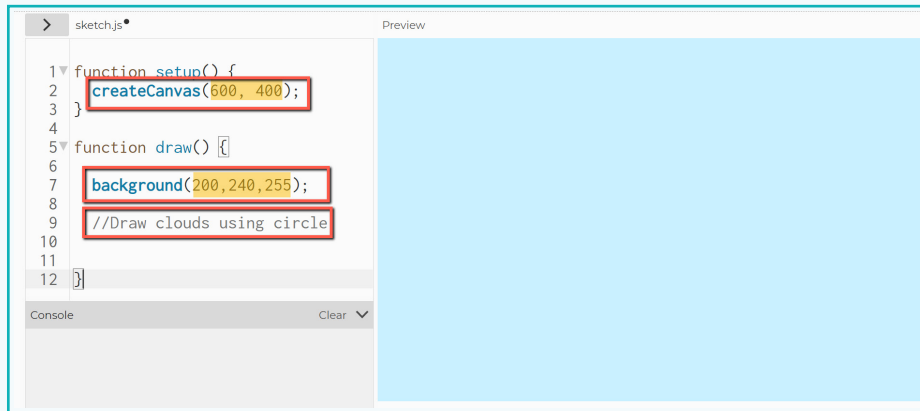
3 Create Background

a. Change the canvas size to 600 by 400 with corresponding values in `createCanvas(600, 400);`

b. To make the sky blue, change the background colour to light blue with `r = 200, g = 240, b = 255`.

Activity

- c. In the code, we can add comments to explain the code snippets, like to draw clouds. Use two forward slashes (//) before text to add comments to the code.

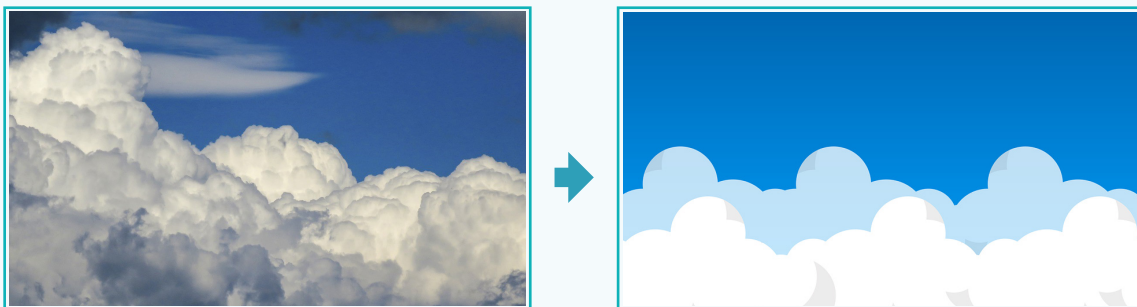


Note:

Comments are additional text added in a program. They are not part of the program; they add document information related to the code. Any text can be made into a comment with two forward slashes '//' before placed before it. This does not affect the code output since compiler does not consider comments a part of code.

4 Draw clouds

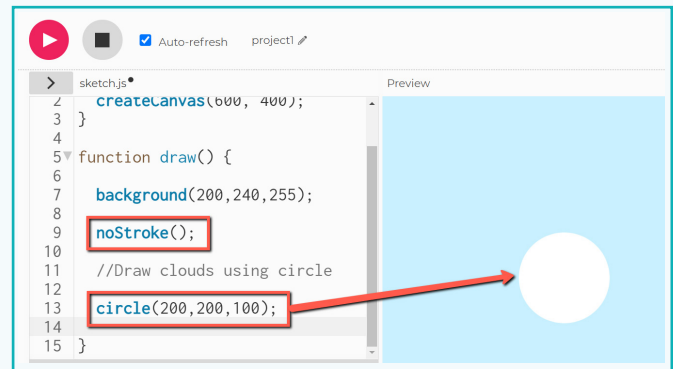
- a. Draw clouds using white circles stacked over each other.



- b. To draw a circle, use command **circle(x, y, diameter)**. x and y show the circle's position on the canvas
- c. The third value in the command decides the circle's diameter.
Place the circle at a position x=200 and y=200 and set diameter 100 pixels. This means that the cloud can be anywhere on the screen.
- d. By default, any shape we draw has a 1-pixel thick black border and white as the shape colour.

Activity

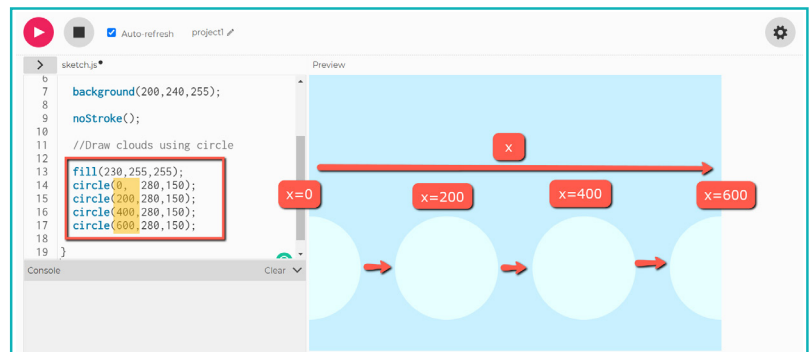
- e. To draw a circle without a border, add command **noStroke()** before **circle()**.



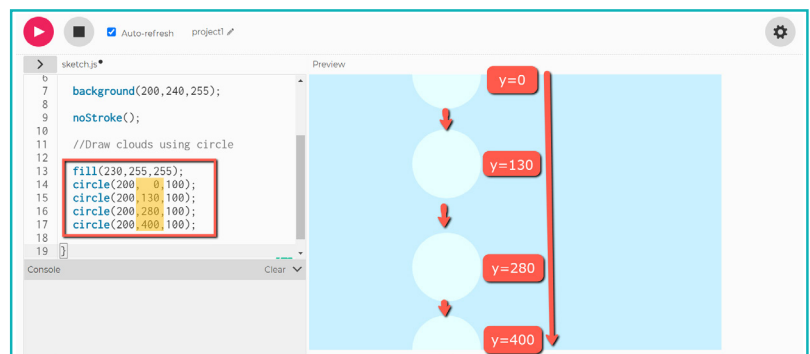
noStroke() is added before any shape command to draw them without a border.

It is important to understand positioning of shapes. As we know, each point on a canvas can be represented with an x and y value. Let's observe how changing values affect a shape's position.

- a. If we add circles with a higher x value, their position moves to the right.



- b. If we add circles with increasing y values on a canvas of height 400, circles will appear from top to bottom.

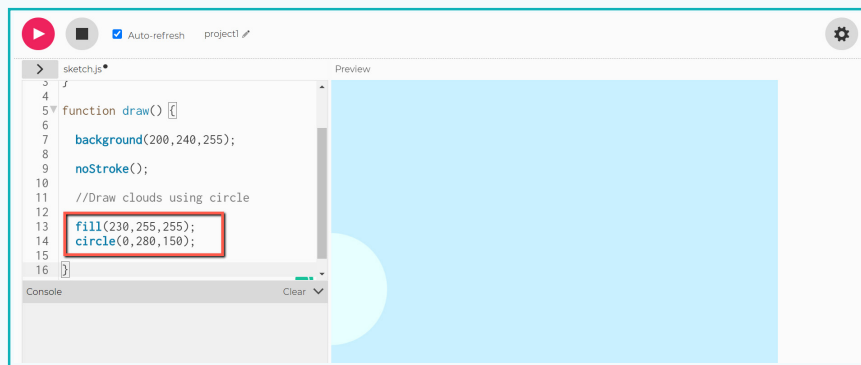


- c. As the x value increases from 0, shape positions move from left to right. As the y value increases from 0, positions move from top to bottom.
- d. If any part of a shape has x,y values higher than the canvas width or height (or less than 0), the shape will move out of the canvas and not be visible respectively.

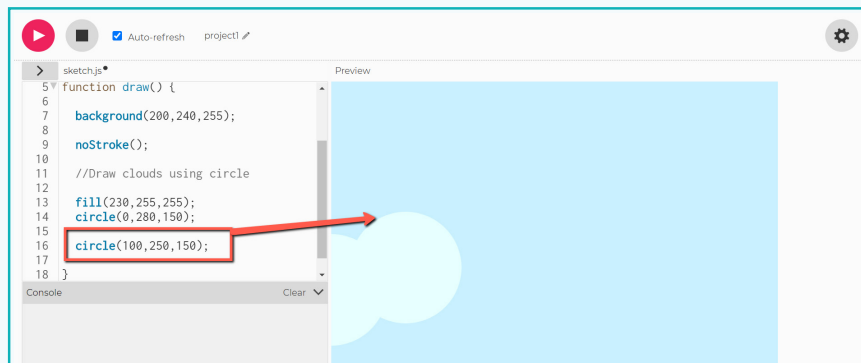
Activity

Add overlapping circles to create clouds on the background.

- Clouds are not pure white. Change their colour to a very light blue with $r = 230$, $g = 255$, $b = 255$.
- Change position of first circle to $x=0$ and $y=280$. Make the diameter 150 to add clouds from the left edge of the screen.

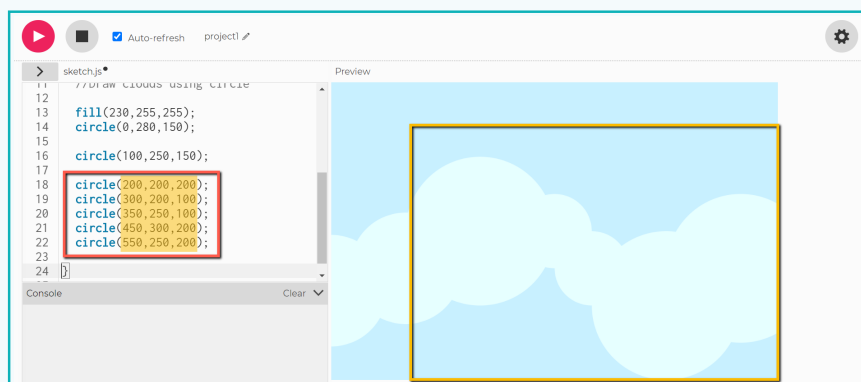


- Add another circle next to the existing one at a slightly higher position (100,250).



Without borders, both circles appear to create a combined single shape.

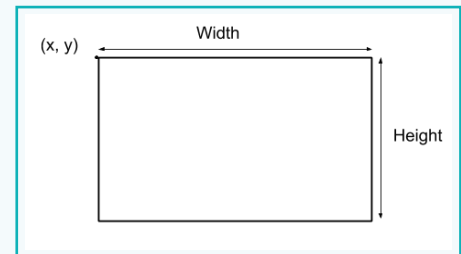
- Add more closely placed circles of different sizes from left to right.



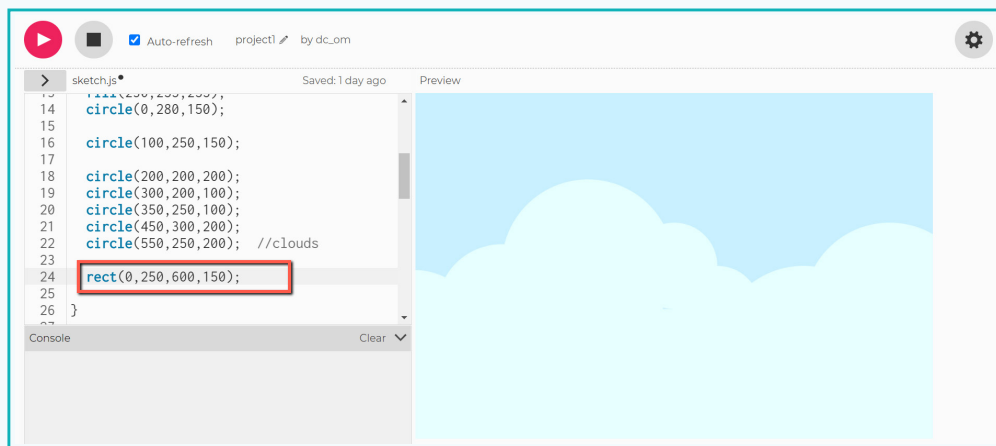
Activity

e. Try changing x,y values and diameters of circles to get different shaped clouds.

f. Add a rectangle to mask the clouds till the canvas's bottom edge. To draw a rectangle, use command `rect(x,y,width,height)`. Here, x and y values represent the top left corner point of the rectangle.



g. Add command `rect(0,250,600,150);`. This will fill all the space from y=250 to the bottom of the canvas. The width is the same as that of the canvas (600) to cover the entire horizontal area.

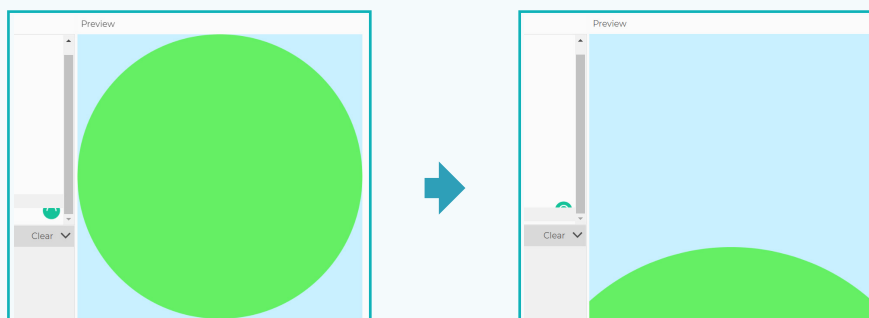


h. To view the rectangle's exact shape, try changing its colour using the fill command. Revert the command later so that it matches with the clouds.

5 Draw mountains

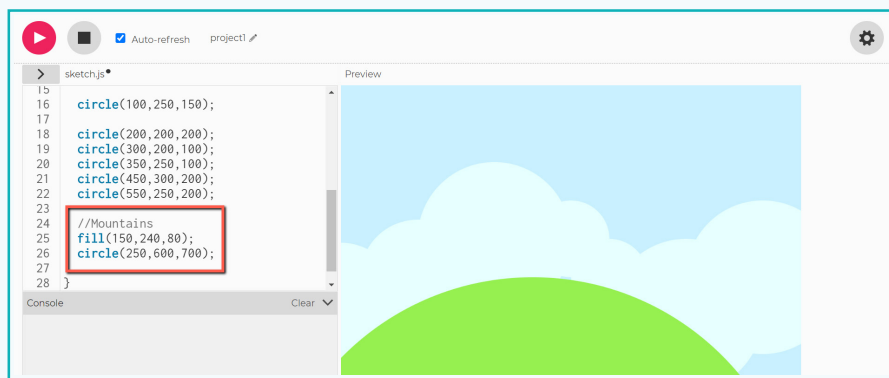
We can show the top part of a circle to make it look like a mountain.

a. Add a circle and increase its size. Move it downwards so that only the upper part can be seen on the canvas.

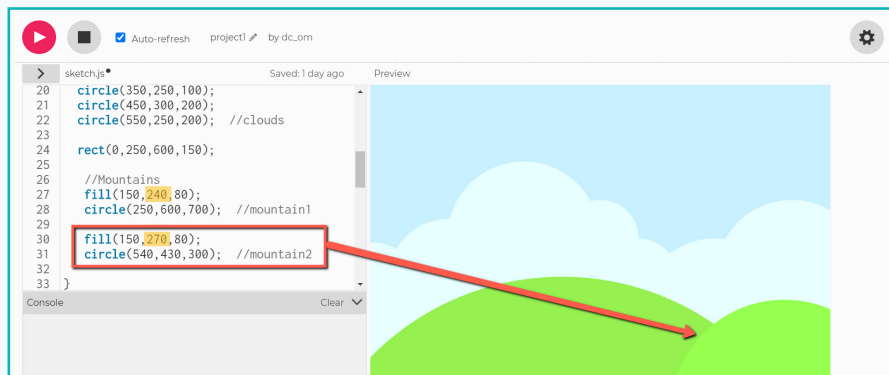


Activity

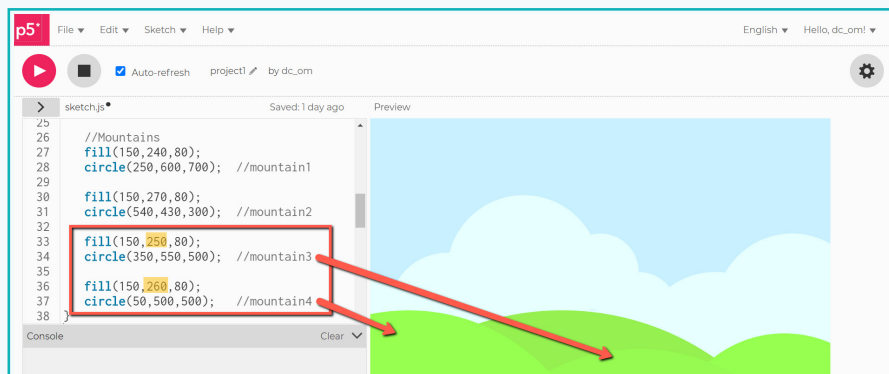
- b. Similarly, add circles to the existing drawing to show mountains.
- c. Add command `fill(150,240,80);` for light green mountains.
- d. Add command `circle(250,600,700);` for a mountain.



- e. Add one more mountain. Change its colour slightly to create a shaded effect. Change the value of g in command `fill(r,g,b)`. Observe the change in colour values.



- f. Add more mountains with slightly different colour shades.
- g. Try out different positions and sizes to create mountains.



Activity

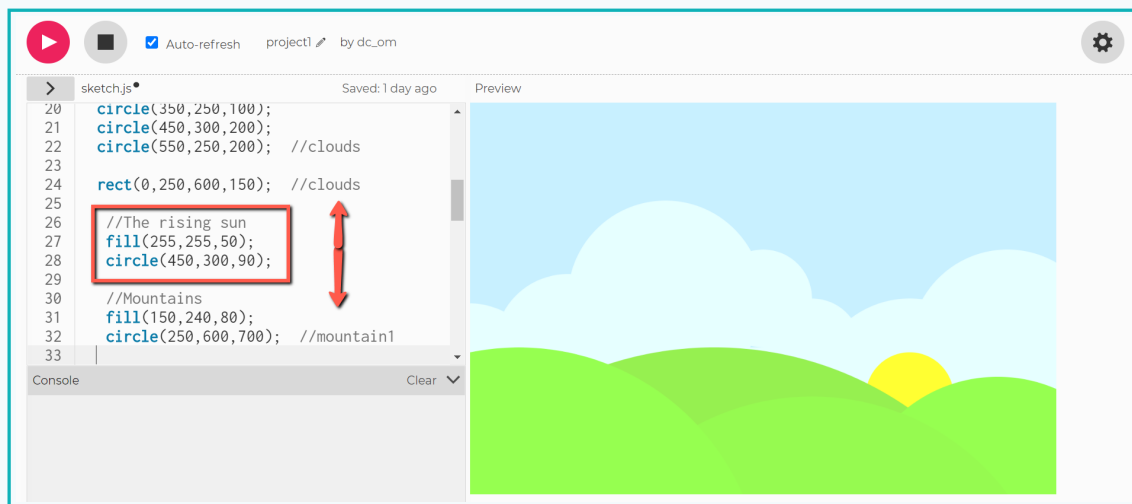
Note:

When shapes are drawn one after the other, the newer shapes overlap earlier ones. This is why it is important to sequence commands.

6 Draw the Sun

Add code to draw the sun. This should be after the code for clouds, but before the code for mountains. This will place the sun behind the mountains, but in front of the clouds.

- Add command `fill(255,255,50);` to colour it yellow.
- Add a circle `(450,300,90);` to place a rising sun between mountains.



Exercise

Answer in one line

- 7 What command can be used to draw a circle at x= 120, y=100 with diameter 50?



- 8 What command can be used to create a canvas measuring 500 by 500 pixels?



- 9 What command can be used to add colour to the background with R=100, G=200, B=255?



- 10 What command is used to draw a shape without an outline or border?



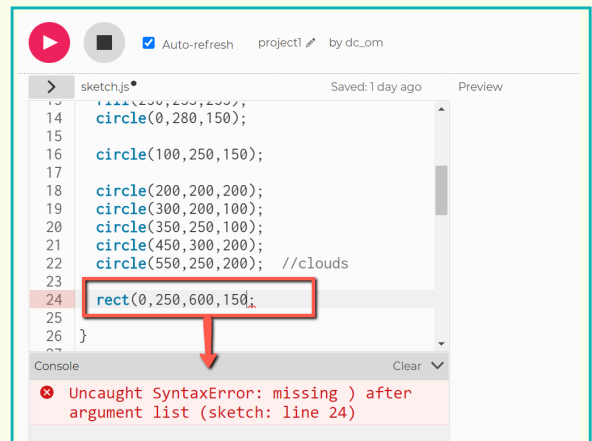
Note:

1. It is important to write each command carefully as command names are case sensitive.

For example: command `circle(200,200,100);` will draw a circle but `Circle(200,200,100);` will not. If a command is correct, its name will turn bold. Try it out and observe.

2. Check every line of code for all the elements of a command. This includes the correct case sensitive name, brackets, number of inputs, comma “,” separating them and semi colon “;” at the end of the command. If we type a wrong code, an error will be shown in the console region.

Observe the example below with a missing bracket.



```
13 sketch.js
14 circle(0,200,200);
15 circle(0,280,150);
16 circle(100,250,150);
17
18 circle(200,200,200);
19 circle(300,200,100);
20 circle(350,250,100);
21 circle(450,300,200);
22 circle(550,250,200); //clouds
23
24 rect(0,250,600,150;.
25
26 }
```

Console

Uncaught SyntaxError: missing) after argument list (sketch: line 24)

We learnt that we could communicate and instruct computers to perform tasks by programming them. Using P5js platform, we explored JavaScript programming language in this lesson. Which are the other programming languages we can learn and use?

There are various ways to create graphics for a digital platform. We have used JavaScript to create Graphics and we will explore it further to make visuals dynamic.



Tech Flash

- **Coding** : Writing a set of instructions in a specific format for computers to perform a function.
- **Programming Language**: A specific coding format with rules and syntax.
- **Binary Language** : A language made of 0s and 1s which a computer understands.
- **Compiler** : A computer program that translates a coding language into binary language.
- **Syntax** : A set of rules for writing a code in a specific programming language.
- **Function** : A set of code which has specific tasks allocated to it. It can be reused in a modular way in the code without repeating its entire code.